

Práctica: Sistema de gestión de ventas (TPV)

1. Contexto

El objetivo de esta práctica es implementar un sistema de gestión de tickets de venta utilizando JDBC. La aplicación permitirá gestionar el inventario, realizar ventas y mantener un histórico de transacciones, asegurando la integridad referencial mediante el uso de claves foráneas y transacciones.

2. Base de Datos

Para la creación de la base de datos utilizamos el script que ya hemos usado durante el desarrollo de la unidad didáctica **(con alguna pequeña modificación)**.

```
CREATE TABLE PRODUCTO(  
  ID NUMBER(9) GENERATED ALWAYS AS IDENTITY NOT NULL PRIMARY KEY,  
  BARCODE VARCHAR2(24 CHAR) NOT NULL,  
  NOMBRE VARCHAR2(200 CHAR) NOT NULL,  
  PRECIO NUMBER(8,2) NOT NULL  
);
```

```
CREATE TABLE TICKET (  
  ID NUMBER(9) GENERATED ALWAYS AS IDENTITY NOT NULL PRIMARY KEY,  
  FECHAHORA TIMESTAMP NOT NULL,  
  TICKETCERRADO VARCHAR2(1) NOT NULL CHECK (TICKETCERRADO IN ('T',  
'F'))  
);
```

```
CREATE TABLE LINEATICKET(  
  ID NUMBER(9) GENERATED ALWAYS AS IDENTITY NOT NULL PRIMARY KEY,  
  CANTIDAD NUMBER(5) NOT NULL,  
  PRECIOVENTA NUMBER(8,2) NOT NULL,  
  PRODUCTO_ID NUMBER(9),  
  TICKET_ID NUMBER(9),  
  FOREIGN KEY(PRODUCTO_ID) REFERENCES PRODUCTO(ID),  
  FOREIGN KEY(TICKET_ID) REFERENCES TICKET(ID) ON DELETE CASCADE  
);
```

3. Estructura del menú

1. **Mantenimiento de productos** (CRUD ya realizado).

2. **Gestión de ventas:**

- 2.1. **Iniciar nueva venta**
- 2.2. **Continuar venta** (Solo si TICKETCERRADO = ' F ').
- 2.3. **Consultar ticket** (Mostramos un ticket a partir de un id solicitado por pantalla. Si el ticket no está cerrado mostramos una advertencia).
- 2.4. **Devolver compra** (Consiste en un borrado de un ticket identificado por su id. Borrado directo gracias al CASCADE. Solo se devolverán compras completas).
- 2.5. **Volver al menú principal.**

0. **Salir.**

4. Requisitos y lógica de negocio

A. Gestión de la venta (Memoria y persistencia)

Para las ventas, se debe trabajar con un objeto Ticket que contenga una List<LineaTicket>.

- **Nueva venta:** Se solicita ID de producto y cantidad. Si el ID se deja en blanco, se pregunta si se desea cerrar el ticket (' T ') o dejarlo abierto para luego (' F ').
- **Transacción:** Al finalizar, el guardado del ticket y todas sus líneas debe ejecutarse bajo una **única transacción** (commit/rollback).

B. Continuar venta

Mostramos un listado de tickets abiertos, ya que solo se puede utilizar esta opción con tickets abiertos (es decir, ticketcerrado = ' F '):

1. Se recuperan los datos del ticket y sus líneas actuales de la base de datos al objeto en memoria y se muestra el estado actual del ticket por pantalla.
2. Se permite añadir más productos siguiendo el mismo flujo que una venta nueva.

C. Restricciones de borrado

- **Tickets:** Gracias al ON DELETE CASCADE, al eliminar un ticket en el menú (opción devolver compra), la base de datos borrará automáticamente sus líneas. No es necesario gestionar transacciones de borrado en Java para este punto.
- **Productos:** No se debe permitir borrar un producto si ya ha sido vendido (si existe en algún ticket que lo incluya en su detalle).

- *Pista:* En lugar de hacer un SELECT previo, intenta borrarlo y captura la excepción de integridad de Oracle (SQLException con el código de error correspondiente).
-

5. Ayudas técnicas

Uso de Timestamp: Para trabajar con la fecha y hora actual en Oracle desde Java:

```
// Para Guardar
pstmt.setTimestamp(n,
java.sql.Timestamp.valueOf(LocalDateTime.now()));
// Para Recuperar
LocalDateTime fecha =
rs.getTimestamp("FECHAHORA").toLocalDateTime();
```

Gestión de Transacciones: Recuerda desactivar el setAutoCommit antes de empezar a guardar el ticket y sus líneas en la base de datos desde los objetos en memoria:

```
try {
    con.setAutoCommit(false);
    // 1. Guardar Ticket y obtener ID
    // 2. Guardar Líneas
    con.commit();
} catch (SQLException e) {
    con.rollback();
    System.out.println("Error en la venta. Operación anulada.");
} finally {
    con.setAutoCommit(true);
}
```

6. Criterios de evaluación

- **Modularidad:** Una clase DAO independiente por cada tabla que implemente su correspondiente interfaz.
- **Integridad:** El programa no debe permitir "tickets huérfanos" si falla la inserción de una línea.
- **Robustez:** Control de errores al intentar borrar productos con histórico de ventas.
- **Limpieza:** Uso correcto de PreparedStatement y cierre de recursos.